VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA1 - HK251

Thiết kế một Iterator

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Mục lục

1 Vấn đề **2**

2 Thiết kế một Iterator **3**

1 Vấn đề

Task sau chúng ta gặp một vấn đề khá nhức nhói là **độ phức tạp**

- **Tên hàm:** `string getContent()`
- **Chức năng:** Trả về toàn bộ nội dung của buffer (danh sách liên kết) dưới dạng một chuỗi.
- **Độ phức tạp:** $O(n)$
- **Ngoại lệ:** Không có.

Nếu sử dụng hàm `get(i)` trong vòng lặp từ 0 đến `size()`, thì độ phức tạp trở thành $O(n^2)$ vì mỗi lần `get(i)` phải duyệt từ đầu đến vị trí i.

```
// O(n*n)
stringstream ss;
for (int i = 0; i < buffer.size(); i++){ // o(n)
    ss << buffer.get(i); // o(n)
}
return ss.str();
```

Chúng ta có 2 cách

1. Giải pháp 1: Duyệt trực tiếp qua `Node`

```
// O(n) traversal using Node*
stringstream ss;
Node* curr = head;
while (curr != nullptr) {
    ss << curr->data;
    curr = curr->next;
}
return ss.str();
```

Ưu điểm:

- Đơn giản, trực tiếp và hiệu quả ($O(n)$).
- Không cần thêm cấu trúc/phụ thuộc mới.

Nhược điểm:

- Truy cập nội bộ: phá vỡ tính đóng gói (encapsulation), vì phải truy cập trực tiếp con trỏ `Node*`.
- Nếu sau này thay đổi cấu trúc liên kết, cần cập nhật nhiều nơi.

2. Giải pháp 2: Duyệt qua `Iterator`

```
// O(n) traversal using iterator
string getContent() const {
    stringstream ss;
    for (char c : *this) ss << c;
    return ss.str();
}
```

Ưu điểm:

- Tuân thủ nguyên lý đóng gói (encapsulation).

- Có thể tận dụng trong nhiều ngữ cảnh khác: `foreach`.
- Dễ dàng bảo trì nếu thay đổi cấu trúc nội bộ.

Nhược điểm:

- Cần hiện thực lớp `Iterator`, có thể phức tạp hơn một chút cho người mới.
- Có thể tốn thêm bộ nhớ nhỏ nếu `Iterator` cần lưu trạng thái.

Giải pháp 1 đơn giản và hiệu quả nhưng phá vỡ đóng gói. Giải pháp 2 sử dụng `Iterator` thân thiện với thiết kế hướng đối tượng hơn, dễ bảo trì về sau. Nếu hệ thống yêu cầu sử dụng nhiều thao tác lặp, giải pháp 2 là lựa chọn nên ưu tiên.

2 Thiết kế một Iterator

- Duyệt qua các phần tử của `LinkedList` bằng vòng lặp `for-each`.
- Có thể thao tác như: truy cập phần tử hiện tại, tiến tới phần tử kế tiếp, so sánh hai iterator.

Bước 1: Định nghĩa giao diện cần hỗ trợ

```
Iterator begin();
Iterator end();
bool operator!=(const Iterator& other) const;
T& operator*() const;
Iterator& operator++();

// có thể thêm
Iterator& operator--();
bool operator==(const Iterator& other) const;
```

Bước 2: Thiết kế lớp Iterator lồng trong LinkedList

```
template <typename T>
class LinkedList {
private:
    struct Node {
        T data;
        Node* next;
    };
    Node* head;

public:
    class Iterator {
    private:
        Node* current;

    public:
        Iterator(Node* node) : current(node) {}

        // Phép so sánh khác
        bool operator!=(const Iterator& other) const {
            return current != other.current;
        }

        // Truy cập dữ liệu tại node hiện tại
        T& operator*() const {
```

```
            return current->data;
        }

        // Tiến đến node tiếp theo
        Iterator& operator++() {
            if (current) current = current->next;
            return *this;
        }
    };

    Iterator begin() const { return Iterator(head); }
    Iterator end() const { return Iterator(nullptr); }

    // Các hàm khác: addLast, remove, v.v...
};
```

Bước 3: Kiểm thử Iterator bằng for (auto x : list)

```
LinkedList<char> list;
list.addLast('a');
list.addLast('b');
list.addLast('c');

for (char c : list) {
    std::cout << c << " ";
}
// Output: a b c
```